

Unit 5- Spring MVC

5.1 Overview of the Spring Framework

What is Spring Framework?

- Spring is a **Java framework** used to develop **enterprise and web applications** in a simple and efficient way.
 - Developed by Rod Johnson
 - Introduced to simplify complex technologies like EJB (Enterprise JavaBeans)
 - It is **lightweight, open-source, and modular**

Simple meaning:

Spring helps developers write **less code** and build applications **faster and easier**

Objectives of Spring:

- Reduce complexity of Java development
- Promote **loose coupling**(means One class **does not depend strongly** on another)
- Improve code **reusability and maintainability**
- Simplify testing and integration

5.2 Spring MVC Annotation

5.3 Spring MVC Architecture

a) Explain spring MVC architecture. And also explain any two spring annotations.

[10]

1. Introduction spring MVC architecture.

- Spring MVC is a **Java web framework** used to develop **web applications** based on the **Model–View–Controller (MVC)** pattern.
- It separates application into:
 - **Model** → Data
 - **View** → Presentation (UI)
 - **Controller** → Business logic
- This separation makes the application:
 - Easy to understand
 - Easy to maintain
 - Scalable

2. Core Components of Spring MVC:

1. DispatcherServlet (Front Controller)

- Central controller of Spring MVC
- Receives **all incoming requests**
- Coordinates entire request flow

Responsibilities:

- Send request to handler mapping
- Call appropriate controller
- Send response to view

2. Handler Mapping

- Maps **URL** → **Controller**
- Helps DispatcherServlet find correct controller

Example:

/home → HomeController

3. Handler Adapter

- Acts as a bridge between **DispatcherServlet** and **Controller**
- Calls controller method properly

Needed because controllers can be of different types

4. Controller

- Processes request
- Contains business logic
- Returns:
 - Model data
 - Logical view name

5. Model

- Holds data (like student details, form data)
- Passed from controller to view

Example:

```
model.addAttribute("name", "Yogita");
```

6. View Resolver

- Converts logical view name → actual view file

Example:

"home" → /WEB-INF/views/home.jsp

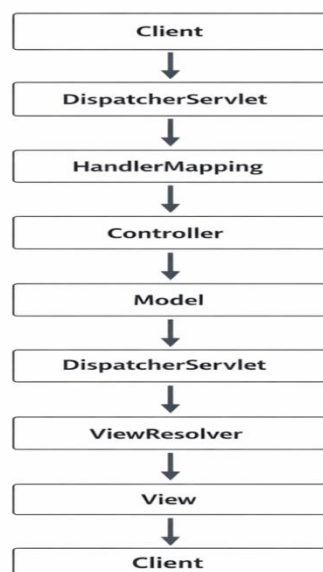
7. View

- Final output shown to user
- Technologies:
 - JSP
 - HTML
 - Thymeleaf

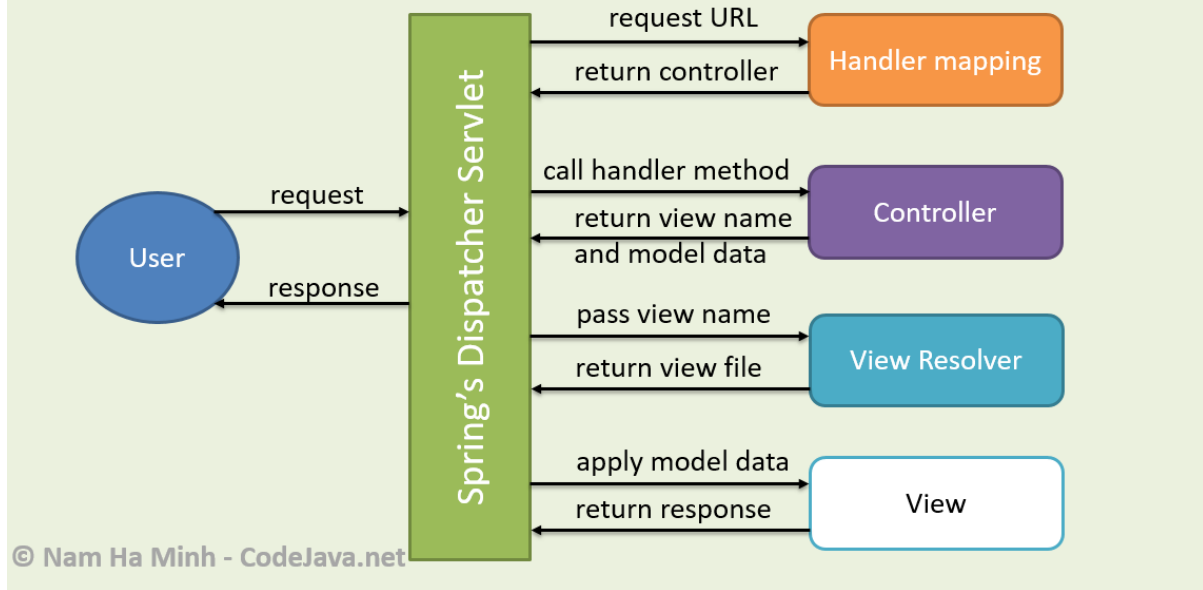
3. Complete Working Flow (Step-by-Step):

1. User sends HTTP request (browser)
2. Request reaches **DispatcherServlet**
3. DispatcherServlet asks **Handler Mapping** for controller
4. Handler Mapping returns controller info
5. DispatcherServlet uses **Handler Adapter**
6. Controller method is executed
7. Controller processes logic
8. Data is stored in **Model**
9. Controller returns logical view name
10. DispatcherServlet sends it to **View Resolver**
11. View Resolver finds actual view (JSP/HTML)
12. View displays response to user

4. Diagram Explanation



Spring MVC's Architecture



5. Advantages of Spring MVC

- Clear separation of concerns
- Reusable components
- Easy testing
- Flexible view technologies
- Scalable architecture

6. Real-Life Example

Online Shopping Website

- User clicks product → Request sent
- Controller fetches product data
- Model stores data
- View displays product page

➤ Spring MVC Annotation

What are Spring MVC Annotations?

- Spring MVC annotations are **special keywords (labels)** used in code to **configure and control the application**.
- Instead of writing XML configuration, we use annotations
Makes code **simple, readable, and fast**

Why use Annotations?

- Reduces XML configuration
- Easy to understand
- Saves development time
- Makes code clean

Important Spring MVC Annotations

1. @Controller

Marks a class as a **Spring MVC Controller**

Example:

```
@Controller
public class HomeController
{

    @RequestMapping("/home")
    public String showHome()
    {
        return "home";
    }
}
```

Meaning:

This class will handle web requests

2. @RequestMapping

- Maps **URL** → **Method**

Example:

```
@RequestMapping("/login")
public String loginPage()
{
    return "login";
}
```

When user enters /login, this method runs

Can also define:

- GET / POST method
- URL pattern

3. @GetMapping

- Shortcut for handling **GET requests**

Example:

```
@GetMapping("/home")
public String home()
{
    return "home";
}
```

4. @PostMapping

- Used for **form submission (POST request)**

Example:

```
@PostMapping("/submit")
public String submitForm()
{
    return "success";
}
```

5. @RequestParam

- Used to get **form data from request**

Example:

```
@GetMapping("/welcome")
public String welcome(@RequestParam String name)
{
    return "welcome";
}
```

Gets value from URL:

/welcome?name=Yogita

Advantages of Annotations

- Less configuration
- Easy coding
- Faster development
- Clean and maintainable code

Real-Life Example

Think annotations like **labels on files**

- You label file → system knows what to do
- Same in Spring → annotation tells Spring how to behave

5.4 Spring MVC Flow

5.6 inversion of control (IoC)

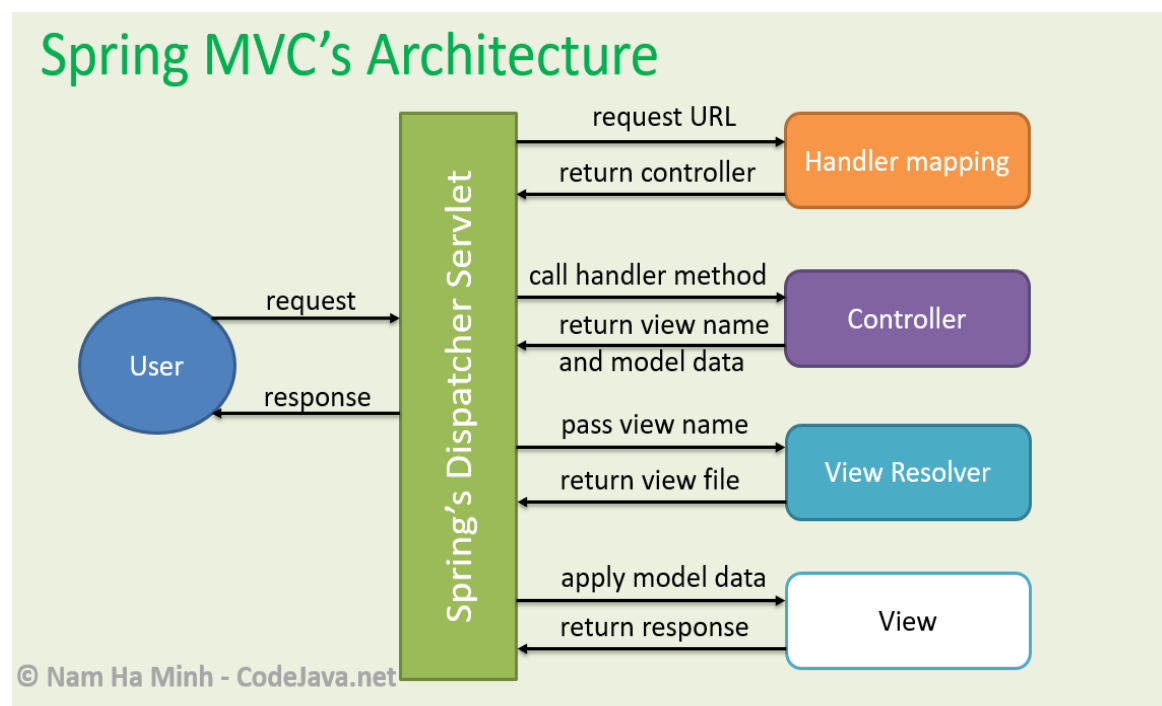
Q. 4 b) Explain Spring mvc flow and inversion of control (IOC)
[10 marks] [Previous year question Jan 2026]

1. Spring MVC Flow:

Introduction

- Spring MVC is a **Java web framework** used to build web applications.
- It follows the **MVC Design Pattern**:
 - **Model** → Data
 - **View** → UI (JSP/HTML)
 - **Controller** → Logic
- It separates application into different parts, making it easy to manage and maintain.

Diagram:



Components of Spring MVC

1. DispatcherServlet

- Central controller (Front Controller)
- Handles all incoming requests
- Controls entire workflow

2. HandlerMapping

- Maps request URL to appropriate controller
- Example: /login → LoginController

3. Controller

- Contains business logic
- Processes user request
- Returns model and view name

4. Model

- Holds data (user info, records, etc.)
- Sent to the view

5. ViewResolver

- Converts logical view name into actual file
- Example: "home" → home.jsp

6. View

- Displays output to user
- Example: JSP, HTML, Thymeleaf page

Step-by-Step Working Flow

1. Client Sends Request

- User enters URL in browser
- Example: http://localhost:8080/home

2. DispatcherServlet Receives Request

- First point of contact
- Acts like a manager

3. DispatcherServlet calls HandlerMapping

- Finds correct controller for request

4. HandlerMapping returns Controller

- Example: HomeController

5. DispatcherServlet calls Controller

- Controller processes request

6. Controller Executes Logic

- Gets data from database/service
- Adds data into Model

7. Controller returns Model + View Name

- Example: ("home", modelData)

8. DispatcherServlet receives response

- Sends it to ViewResolver

9. ViewResolver finds actual View

- Example: home.jsp

10. View Displays Data

- Data shown to user
- Final output appears in browser

2. Inversion of Control (IoC) :

What is IoC?

Definition:

- Inversion of Control (IoC) means the Spring framework creates and manages objects instead of the programmer.
- This makes the code simpler, loosely coupled, and easier to maintain.

Simple Meaning

Normally:

- Programmer creates objects

In IoC:

- Spring creates and manages objects

That's why it is called **“Inversion (reverse) of Control”**

Traditional Way of create object (Without IoC)

```
Car c = new Car();
```

✗Problems:

- Tight coupling
- Hard to maintain
- Difficult to test

Here programmer controls everything

Spring Way of create object (With IoC)

```
// Spring creates object automatically  
Car c; // no 'new' keyword
```

- Here Spring creates object
- Programmer does not create object

What Control is Inverted?

In IoC, the following controls are transferred to Spring:

- Object creation
- Object lifecycle
- Dependency management
- Configuration

IoC Container

- IoC container is the **main component of Spring**
- **It is responsible for:**
 - Creating objects (called Beans)
 - Injecting dependencies
 - Managing lifecycle
 - Configuring objects

Types of IoC Container

1. BeanFactory

- BeanFactory is the **basic IoC container** in Spring used to **create and manage objects (beans)**.
- **Key Points:**
 - **Basic container**
 - Simple and lightweight
 - Provides only basic features of Spring
 - **Uses Lazy Loading**
 - Objects are created **only when needed**
 - Saves memory
 - **Less Features**
 - Mostly used in small applications

2. ApplicationContext

- ApplicationContext is an **advanced IoC container** that provides **all features of BeanFactory + extra features**.
- **Key Points**
 - Advanced container
 - More features (recommended)
 - Used in most applications

5.6 Spring Core and Spring Boot – Dependency Injection (DI)

Introduction to Dependency Injection (DI)

Definition:

- Dependency Injection (DI) is a concept in Spring where **objects are provided (injected) by the Spring container instead of creating them manually.**
- It is used to achieve **Inversion of Control (IoC)**

Simple Meaning

Normally:

- We create objects using new

In Spring:

- Spring **creates and injects objects automatically**

Example Without DI (Traditional Way)

```
Engine engine = new Engine();  
Car car = new Car(engine);
```

✗Problems:

- Tight coupling (means Classes are strongly dependent on each other).
- Difficult to maintain

Example With DI (Spring Way)

```
@Autowired  
Engine engine;
```

- ✓Spring injects dependency automatically
- ✓Loosely coupled

What is @Autowired?

- @Autowired is an annotation in Spring Framework used for **Dependency Injection (DI)**.
- It tells Spring:

“Please automatically provide (inject) the required object here.”

What happens internally?

- Instead of you writing:

```
Engine engine = new Engine(); // manual creation
```

- Spring does this automatically:
 - Creates Engine object
 - Injects it into your class

You don't control object creation
Spring container controls it (IoC)

Spring Core supports DI in 2 ways:

1. XML Configuration (Old Method)

Code:

```
<bean id="engine" class="Engine"/>  
  
<bean id="car" class="Car">  
  <property name="engine" ref="engine"/>  
</bean>
```

Explanation:

- <bean> → defines object
- id="engine" → name of object

- `class="Engine"` → class name

This creates an **Engine object**

- `<property name="engine" ref="engine"/>`

Means:

- Inject engine bean into car

Simple Meaning:

- All dependencies are written in **XML file**
- Spring reads XML and creates + connects objects

✗ Drawbacks:

- Too much XML (lengthy)
- Hard to read and maintain
- Not modern

2. Annotation-Based Configuration (Modern Method)

Code:

```
@Component
class Engine
{
}
```

```
@Component
class Car
{
    @Autowired
    Engine engine;
}
```

Explanation:

- @Component → tells Spring to create object (bean)
- @Autowired → inject dependency automatically

Simple Meaning:

- No XML required
- Everything done using **annotations in Java code**
- Spring automatically:
 - Creates objects
 - Injects dependencies

➤ Why Annotation is Better?

XML Method ✗ Annotation Method ✓

More code Less code

XML file needed No XML

Hard to maintain Easy to maintain

Old approach Modern approach

➤ Features of DI in Spring Boot

- No XML configuration required
- Uses annotations only
- Auto-configuration available
- Faster development
- Easy to use

➤ **Common Annotations Used**

- @Component → General bean
- @Service → Service layer
- @Repository → Database layer
- @Controller → Web layer
- @Autowired → Inject dependency

➤ **Advantages of Dependency Injection**

- Loose coupling
- Easy testing
- Better code management
- Reusability
- Easy maintenance

➤ **Key Points**

- DI = Injecting objects automatically
- Part of IoC
- Spring Core → XML + Annotation
- Spring Boot → Mostly Annotation
- Reduces dependency between classes

5.5 Spring Form Handling

Introduction

- Spring Form Handling is used to:
- **Take input from user through a form**
- **Process that data in backend (Controller)**
- **Display result to user**

✓ It is commonly used in:

- Registration forms
- Login forms
- Feedback forms

Simple Meaning

User fills form → Data goes to controller → Process → Result shown

Flow of Form Handling

User → Form (HTML/JSP) → Controller → Model → View → User

Steps in Spring Form Handling

Step 1: Create Model Class (Data Holder)

- This class stores form data
- Example:

```
public class Student
{
    private String name;
    private String email;

    // getters and setters
}
```

✓ Holds user input values

Step 2: Create Controller

- Controller handles request and form data.
- Example:

```
@Controller
public class StudentController
{

    @GetMapping("/form")
    public String showForm(Model model)
    {
        model.addAttribute("student", new Student());
        return "form";
    }

    @PostMapping("/submit")
    public String submitForm(@ModelAttribute Student student)
    {
        return "result";
    }
}
```

Explanation

- @GetMapping("/form")
 - Loads the form page
- model.addAttribute()
 - Sends empty object to form
- @PostMapping("/submit")
 - Handles form submission
- @ModelAttribute
 - Automatically binds form data to object

Step 3: Create Form (View Page)

```
<form action="/submit" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="text" name="email"><br>
  <input type="submit" value="Submit">
</form>
```

✓ User enters data here

Step 4: Result Page

```
<h2>Form Submitted Successfully</h2>
```

It Displays output.

➤ How Data Binding Works

Spring automatically maps:

- Form field name → Object variable

Example:

- name="name" → student.name
- name="email" → student.email

✓ This is called **Data Binding**

➤ Validation in Form Handling

- Used to check user input (important in exam)
- Example:

```
public class Student
```

```
{  
  
    @NotEmpty  
    private String name;  
  
    @Email  
    private String email;  
}
```

➤ Controller with Validation

```
@PostMapping("/submit")  
public String submitForm(@Valid @ModelAttribute Student student,  
                        BindingResult result)  
{  
  
    if(result.hasErrors())  
    {  
        return "form";  
    }  
  
    return "result";  
}
```

Explanation

- @Valid → Enables validation
- BindingResult → Checks errors
- If error → show form again

Important Annotations

- @Controller → Defines controller
- @GetMapping → Load form
- @PostMapping → Submit form
- @ModelAttribute → Bind data
- @Valid → Validate data

➤ **Advantages of Spring Form Handling**

- Automatic data binding
- Easy validation
- Clean code
- Less boilerplate code
- Better user input handling

➤ **Real-Life Example**

Online registration form:

- User enters name, email
- System checks data
- Stores and displays success message

Example 1:

Q.5) b) Create a spring MVC form to read registration details for Blood donation camp with spring validation & display it.

[10 marks]

[Previous year question paper May 2025]

STEP 1: Create Spring Boot Project

1. Open IntelliJ
2. Click **New Project**
3. Select **Spring Boot**
4. Fill:
 - Name: bloodcamp
 - Language: Java
 - Packaging: Jar
5. Add Dependencies:
 - ✓ Spring Web
 - ✓ Thymeleaf
 - ✓ Validation

Click **Finish**

STEP 2: Project Structure (Auto Created)

You will see:

src/main/java
src/main/resources

STEP 3: Create Model Class

1. Create Package

- Right click on com.blood.bloodcamp
- Click **New** → **Package**
- Name it: model

2. Create Java Class

- Right click on model
- Click **New** → **Java Class**
- Name: Donor

Code: Donor.java

```
package com.blood.bloodcamp.model;

import jakarta.validation.constraints.*;

public class Donor
{

    @NotEmpty(message="Name is required")
    private String name;

    @Email(message="Invalid email")
    private String email;

    @Min(value=18, message="Minimum age is 18")
    private int age;

    @NotEmpty(message="Blood group required")
    private String bloodGroup;

    // Getters and Setters
    public String getName()
    { return name; }
    public void setName(String name)
```

```

{ this.name = name; }

public String getEmail()
{ return email; }
public void setEmail(String email)
{ this.email = email; }

public int getAge()
{ return age; }
public void setAge(int age)
{ this.age = age; }

public String getBloodGroup()
{ return bloodGroup; }
public void setBloodGroup(String bloodGroup)
{ this.bloodGroup = bloodGroup; }
}

```

STEP 4: Create Controller

1. Create Package

- Right click on com.blood.bloodcamp
- Click **New** → **Package**
- Name it: controller

2. Create Java Class

- Right click on model
- Click **New** → **Java Class**
- Name: DonorController

Code: DonorController.java

```

package com.blood.bloodcamp.controller;

import com.blood.bloodcamp.model.Donor;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;

```

```

import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
public class DonorController {

    @GetMapping("/")
    public String showForm(Model model) {
        model.addAttribute("donor", new Donor());
        return "form";
    }

    @PostMapping("/submit")
    public String submitForm(@Valid @ModelAttribute("donor") Donor donor,
        BindingResult result) {

        if(result.hasErrors()) {
            return "form";
        }

        return "result";
    }
}

```

STEP 5: Create HTML Pages

Path: src/main/resources/templates

Code: form.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Blood Donation Form</h2>

<form th:action="@{/submit}" th:object="${donor}" method="post">

    Name: <input type="text" th:field="*{name}"/>
    <span th:errors="*{name}"></span><br><br>

```

```
Email: <input type="text" th:field="*{email}"/>
<span th:errors="*{email}"></span><br><br>

Age: <input type="text" th:field="*{age}"/>
<span th:errors="*{age}"></span><br><br>

Blood Group: <input type="text" th:field="*{bloodGroup}"/>
<span th:errors="*{bloodGroup}"></span><br><br>

<button type="submit">Submit</button>

</form>

</body>
</html>
```

Code: result.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Submitted Details</h2>

Name: <span th:text="${donor.name}"></span><br>
Email: <span th:text="${donor.email}"></span><br>
Age: <span th:text="${donor.age}"></span><br>
Blood Group: <span th:text="${donor.bloodGroup}"></span><br>

</body>
</html>
```

STEP 6: Run Project

1. Open main file:
BloodcampApplication.java
2. Click **Run** ►

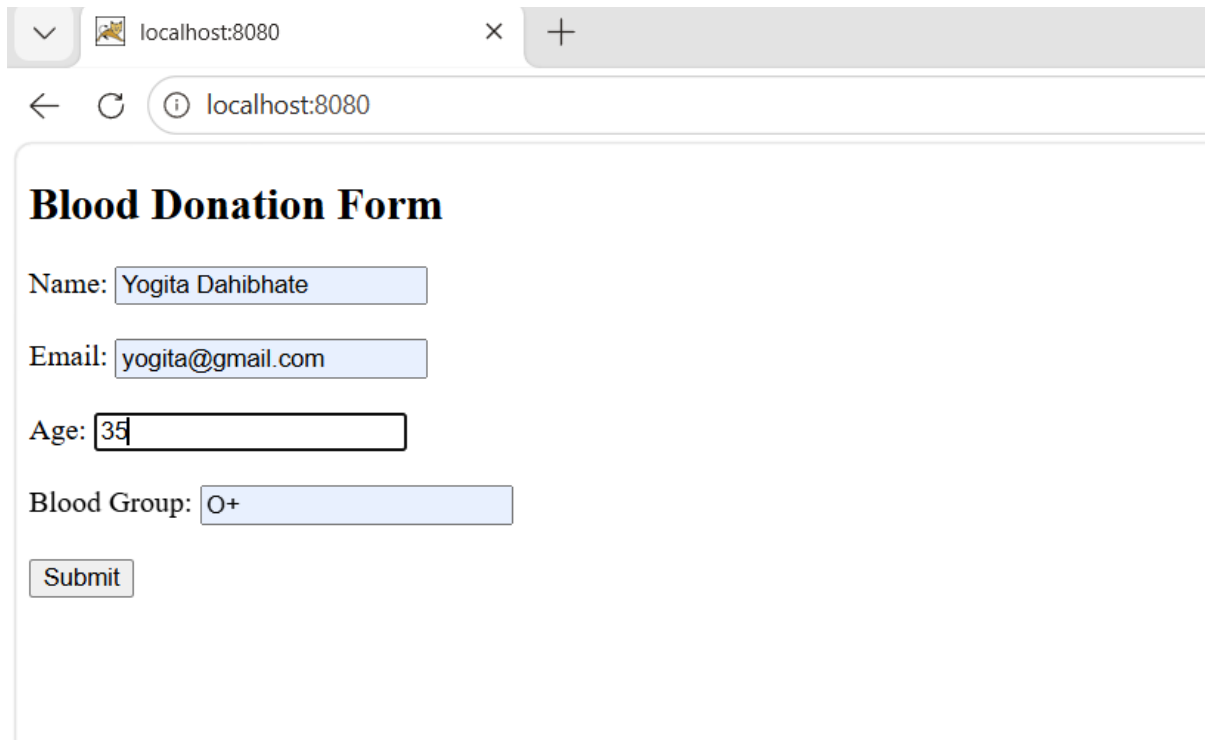
STEP 7: Open in Browser

Go to:

<http://localhost:8080/>

OUTPUT

- Form will open
- Enter data
- Validation works
- Data displayed on next page



The screenshot shows a web browser window with the address bar displaying 'localhost:8080'. The page title is 'Blood Donation Form'. The form contains the following fields and values:

- Name:
- Email:
- Age:
- Blood Group:
- Submit:

Submitted Details

Name: Yogita Dahibhate
Email: yogita@gmail.com
Age: 35
Blood Group: O+

Validation:

Blood Donation Form

Name: Name is required

Email: Invalid email

Age: Minimum age is 18

Blood Group:

Example 2:

Q.5) a) Create a MVC form to read registration details for Placement Drive with Spring Validation & display it. [10 Marks]

[Previous year Question paper JAN 2026]

STEP 1: Create Spring Boot Project

1. Open IntelliJ
2. Click **New Project**
3. Select **Spring Boot**

Fill details:

- Name: placementdrive
- Language: Java
- Packaging: Jar

Add Dependencies:

- ✓ Spring Web
- ✓ Thymeleaf
- ✓ Validation

Click Create/ finish

STEP 2: Project Structure (Auto Created)

You will see:

- src/main/java
- src/main/resources

STEP 3: Create Model Class

1. Create Package

- Right click on com.placement.placementdrive
- New → Package
- Name: model

2. Create Java Class

- Right click on model
- New → Java Class
- Name: Candidate

Code: Candidate.java

```
package com.placement.placementdrive.model;

import jakarta.validation.constraints.*;

public class Candidate
{

    @NotEmpty(message="Name is required")
    private String name;

    @Email(message="Invalid email")
    private String email;

    @Min(value=18, message="Minimum age is 18")
    private int age;

    @NotEmpty(message="Qualification required")
    private String qualification;

    // Getters and Setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
```

```

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public int getAge() { return age; }
public void setAge(int age) { this.age = age; }

public String getQualification() { return qualification; }
public void setQualification(String qualification) { this.qualification =
qualification; }
}

```

STEP 4: Create Controller

1. Create Package

- Right click on com.placement.placementdrive
- New → Package
- Name: controller

2. Create Java Class

- Name: CandidateController

Code: CandidateController.java

```

package com.placement.placementdrive.controller;

import com.placement.placementdrive.model.Candidate;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
public class CandidateController
{

    @GetMapping("/")

```

```

public String showForm(Model model)
{
    model.addAttribute("candidate", new Candidate());
    return "form";
}

@PostMapping("/submit")
public String submitForm(@Valid @ModelAttribute("candidate") Candidate
candidate,
                        BindingResult result)
{
    if(result.hasErrors())
    {
        return "form";
    }

    return "result";
}
}

```

STEP 5: Create HTML Pages

Path: src/main/resources/templates

Code: form.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Placement Drive Registration</h2>

<form th:action="@{/submit}" th:object="${candidate}" method="post">

    Name: <input type="text" th:field="*{name}"/>
    <span th:errors="*{name}"></span><br><br>

    Email: <input type="text" th:field="*{email}"/>
    <span th:errors="*{email}"></span><br><br>

    Age: <input type="text" th:field="*{age}"/>

```

```
<span th:errors="*{age}"></span><br><br>

Qualification: <input type="text" th:field="*{qualification}" />
<span th:errors="*{qualification}"></span><br><br>

<button type="submit">Submit</button>

</form>

</body>
</html>
```

Code: result.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Submitted Details</h2>

Name: <span th:text="{candidate.name}"></span><br>
Email: <span th:text="{candidate.email}"></span><br>
Age: <span th:text="{candidate.age}"></span><br>
Qualification: <span th:text="{candidate.qualification}"></span><br>

</body>
</html>
```

STEP 6: Run Project

- Run PlacementdriveApplication.java
- Click ► Run

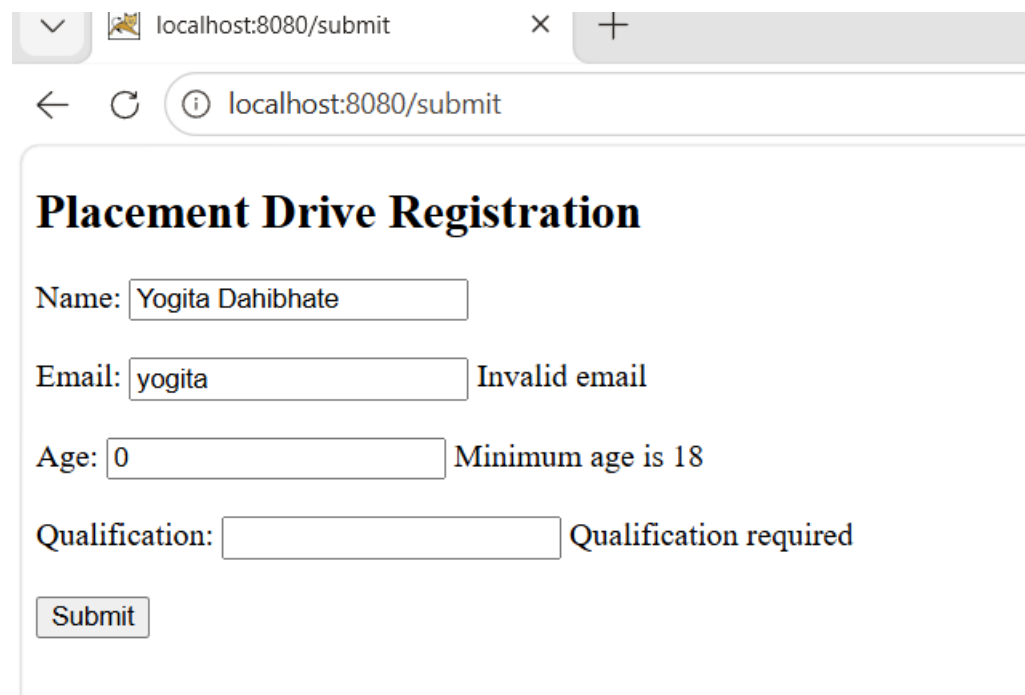
STEP 7: Open in Browser

<http://localhost:8080/>

OUTPUT:

- Form will open
- Enter details
- Validation will check input
- Data displayed on next page

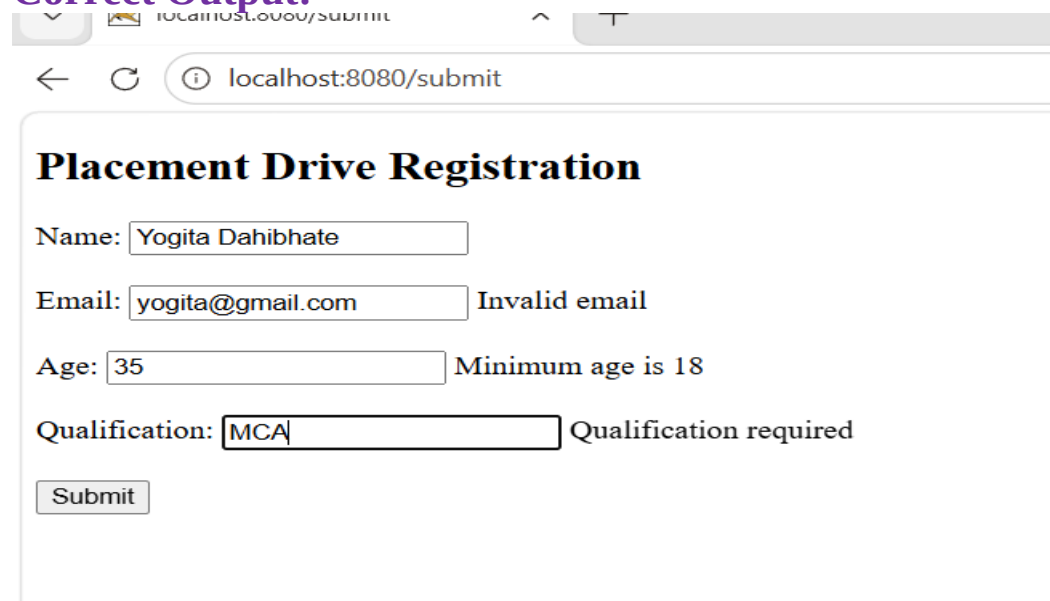
Validation Form:



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/submit'. The page title is 'Placement Drive Registration'. The form contains the following fields and validation messages:

- Name:
- Email: Invalid email
- Age: Minimum age is 18
- Qualification: Qualification required
-

Correct Output:



localhost:8080/submit

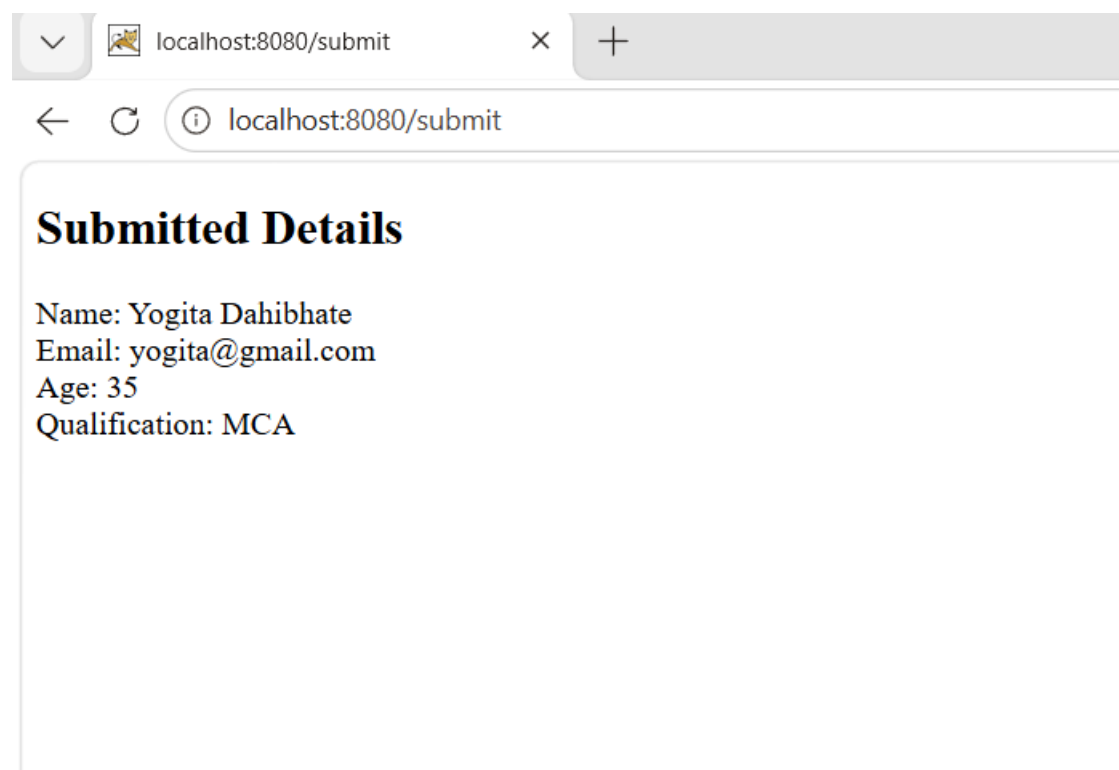
Placement Drive Registration

Name:

Email: Invalid email

Age: Minimum age is 18

Qualification: Qualification required



localhost:8080/submit

Submitted Details

Name: Yogita Dahibhate
Email: yogita@gmail.com
Age: 35
Qualification: MCA